

Generating Runtime-Verification Specs from Natural Language



Daniel Feng

Systems Lab 2026
YiImaz

Introduction

Runtime Verification (RV) is the software engineering process of continuously monitor execution traces and catch logical violations in real-time. Runtime Verification (RV) catches logical violations in real-time—but requires writing complex formal logic that few engineers know. However, translating simple English requirements into the complex formal logic required by RV frameworks makes a high barrier-of-entry to RV. To address this bottleneck, this project automates the translation of natural language (NL) software requirements into Extended Regular Expressions (ERE) for the Monitoring-Oriented Programming (MOP) framework by utilizing Large Language Models. Mop example is in Fig. 1.



Fig. 1: Labeled MOP File Example

Novelty

While prior research (nl2spec, Req2LTL) has explored using LLMs to translate natural language into formal logic, they focus on logic languages other than ERE (focusing on Linear Temporal Logic instead) and aren’t tuned for the context of runtime-verification (the input of not only NL but also Params,

Events, and Handler). Additionally, nl2spec relied on human experts to verify the accuracy of the translation tool, while we developed an automated semantic comparison engine that rigorously defines what “equivalence” means.

Methodology

We began with building the ERE Generator (EREG). We iteratively developed a prompt, and then tried multiple LLMs to be the foundational model (GPT 5.5, OSS-120b, OSS-120b fine-tuned on RegEx databases). To further improve results, we also tried a multi-transformer architecture, consisting of an NL Clarifier (NLC) and Event Interpreter (EI, or Event-to-NL), before the EREG. The NLC improved the structure of the NL and added context, which was lacking in the raw NL. The EI translated the “Events” section’s code into English, which when ported into the ERE Generator improved understanding of the Events. Full tool diagram is in Fig. 2. Finally, we developed a Python engine that converts both the generated and ground-truth EREs into modified minimal deterministic finite automata (min-DFA) to mathematically prove their semantic equivalence. We tested the tool on a database of Javadoc MOP specifications, example seen in Fig. 1, removing the logic and passing the rest of the file into the tool.

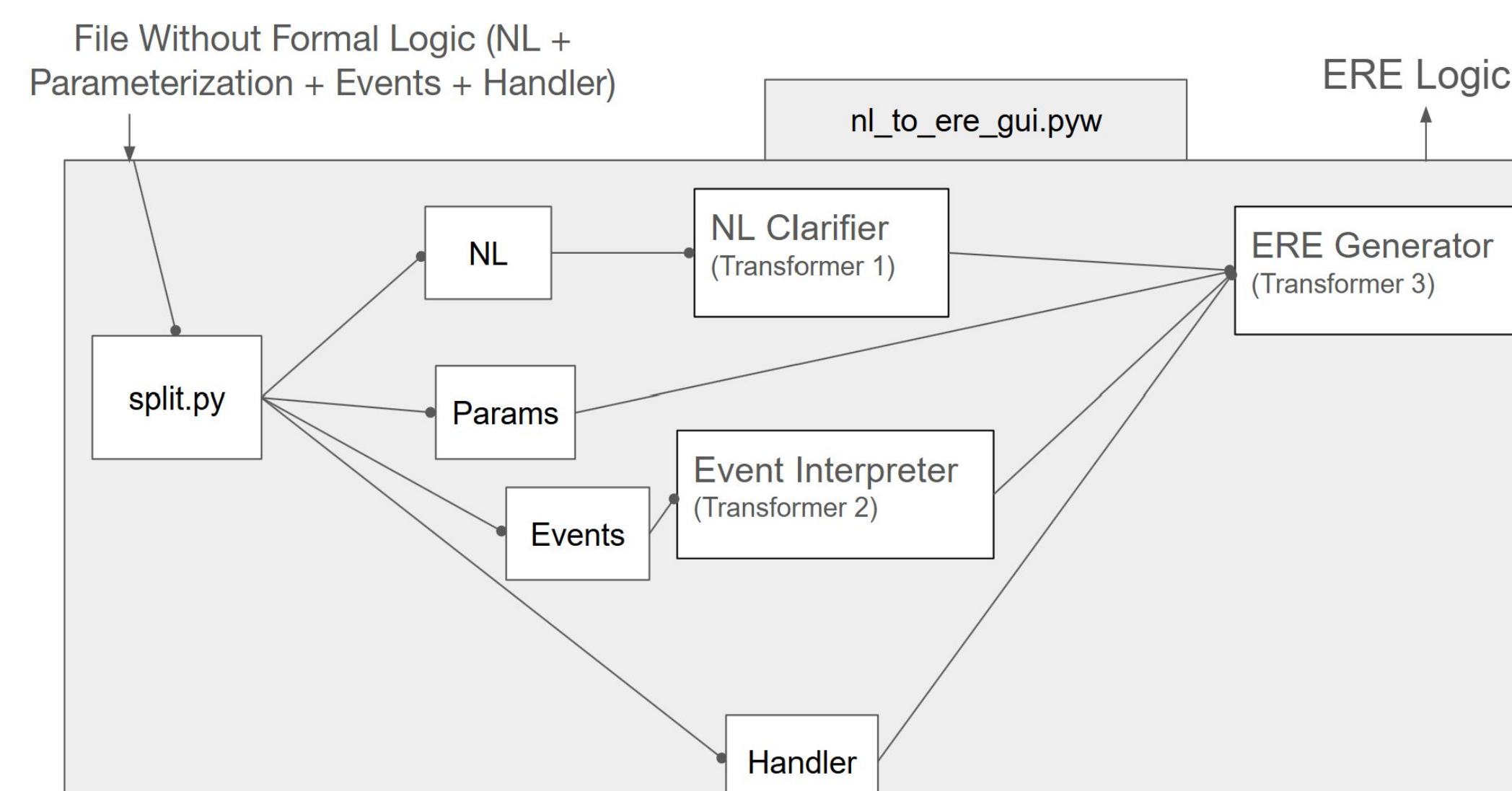


Fig. 2: Full Tool Architecture

Results

Model Configuration	Semantic	Syntactic
GPT 5.5 w/ Iterative Prompt	0.42	0.32
GPT 5.5 w/ Iterative Prompt & NL Clarifier & Event-to-NL	0.61	0.44
GPT-OSS w/ Iterative Prompt	0.22	0.15
GPT-OSS w/ Iterative Prompt & NL Clarifier & Event-to-NL	0.28	0.19
Fine-tuned GPT-OSS w/ Iterative Prompt & NL Clarifier & Event-to-NL	0.35	0.21

Table 1: Results of Multiple Architectures

As seen in Table 1, Our best model was GPT 5.5 with all addons, receiving a semantic accuracy of 0.61. The fine-tuned GPT-OSS models (and non-fine tuned) performed very poorly compared to the flagship GPT model. The main errors seen across all models were forgetting ERE syntax (specifically attempting to use RegEx only syntax) and logical errors (failing to catch some valid/invalid pattern), and these errors were much more common in the GPT-OSS architectures. The NL Clarifier & Event-to-NL tools did seem to significantly improve the accuracy, from 0.42 to 0.61, but even with them GPT-OSS did not surpass GPT 5.5 without them.

Impact & Future Work

The automation of ERE logic generation significantly lowers the barrier to entry for using MOP and also saves time even for experienced software engineers; the tool turns ERE generation time from hours to seconds. Future iterations will explore capabilities for other formal logics, as evidence shows that logic like Linear Temporal Logic may be easier for transformer models to interpret. We also plan to test the strength of this tool on industry runtime-verification tasks, as the testing done for this project was on Javadoc MOP specifications. Lastly, we plan to open-source all architectures and comparison tool.

Generating Runtime-Verification Specs from Natural Language

Daniel Feng



Introduction

Runtime Verification (RV) is the software engineering process of continuously monitor execution traces and catch logical violations in real-time. Runtime Verification (RV) catches logical violations in real-time—but requires writing complex formal logic that few engineers know. However, translating simple English requirements into the complex formal logic required by RV frameworks makes a high barrier-of-entry to RV. To address this bottleneck, this project automates the translation of natural language (NL) software requirements into Extended Regular Expressions (ERE) for the Monitoring-Oriented Programming (MOP) framework by utilizing Large Language Models.



Fig. 1: Labeled MOP File Example

Novelty

While prior research (nl2spec, Req2LTL) has explored using LLMs to translate natural language into formal logic, they focus on logic languages other than ERE (focusing on Linear Temporal Logic instead) and aren't tuned for the context of runtime-verification (the input of not only NL but also Params,

Events, and Handler). Additionally, nl2spec relied on human experts to verify the accuracy of the translation tool, while we developed an automated semantic comparison engine that rigorously defines what “equivalence” means.

Methodology

We began with building the ERE Generator (EREG). We iteratively developed a prompt, and then tried multiple LLMs to be the foundational model (GPT 5.5, OSS-120b, OSS-120b fine-tuned on RegEx databases). To further improve results, we also tried a multi-transformer architecture utilizing, consisting of an NL Clarifier (NLC) and Event Interpreter (EI, or Event-to-NL), before the EREG. The NLC improved the structure of the NL and added context, which was lacking in the raw NL. The EI translated the “Events” section’s code into English, which when ported into the ERE Generator improved understanding of the Events. Diagram is in Fig. 2. Finally, we developed a Python engine that converts both the generated and ground-truth EREs into modified minimal deterministic finite automata (min-DFA) to mathematically prove their semantic equivalence. We tested the tool on a database of Javadoc MOP specifications, example seen in Fig. 1, removing the logic and passing the rest of the file into the tool.

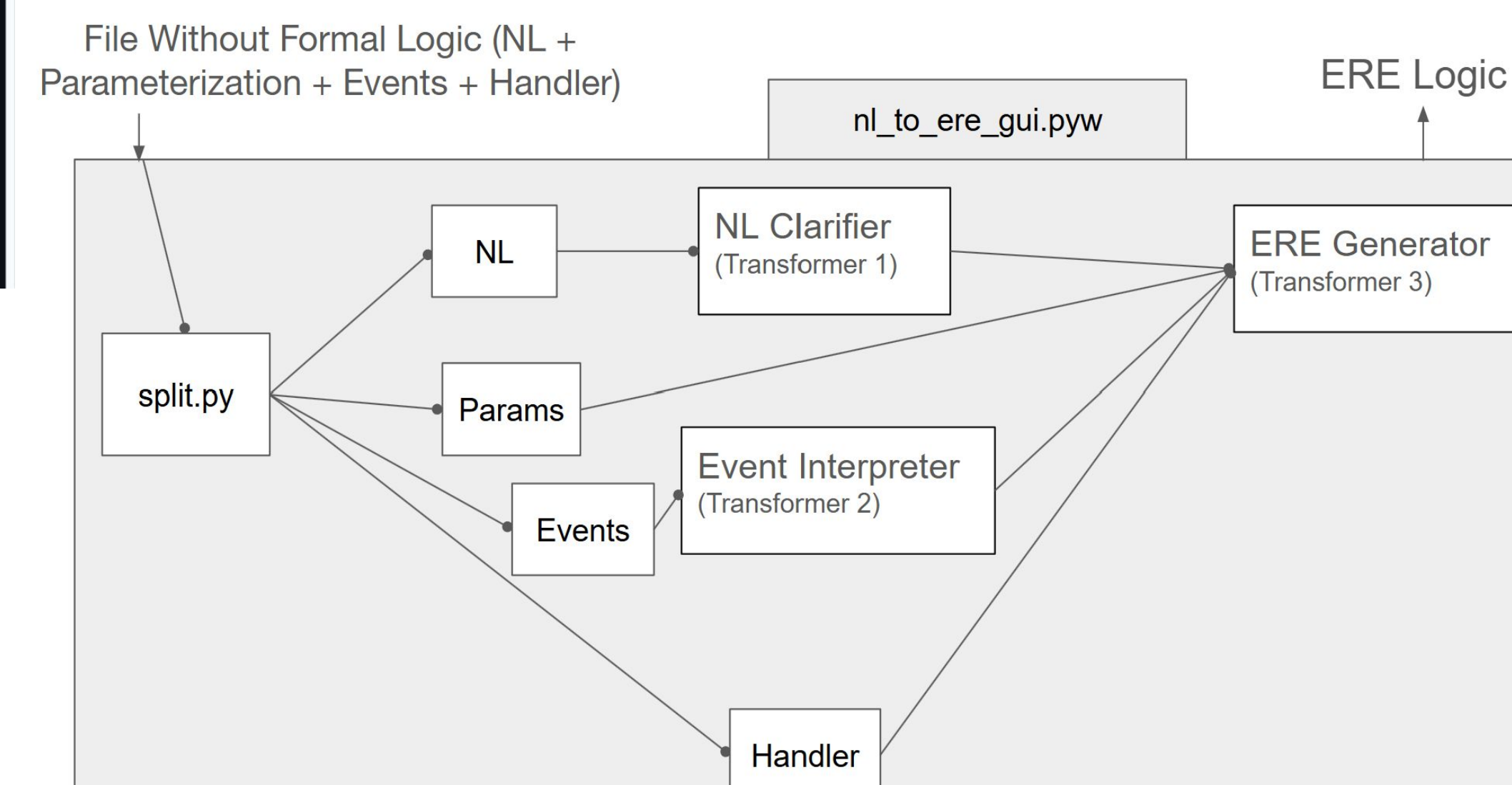


Fig. 2: Full Tool Architecture

Results

Performance of Evaluated Models on ERE Generation		
Model Configuration	Semantic Accuracy	Syntactic Accuracy
GPT 5.5 w/ Iterative Prompt	0.42	0.32
GPT 5.5 w/ Iterative Prompt & NL Clarifier & Event-to-NL	0.61	0.44
GPT-OSS w/ Iterative Prompt	0.22	0.15
GPT-OSS w/ Iterative Prompt & NL Clarifier & Event-to-NL	0.28	0.19
Fine-tuned GPT-OSS w/ Iterative Prompt & NL Clarifier & Event-to-NL	0.35	0.21

Table 1: Results of Multiple Architectures

As seen in Table 1, Our best model was GPT 5.5 with all addons, receiving a semantic accuracy of 0.61. The fine-tuned GPT-OSS models (and non-fine tuned) performed very poorly compared to the flagship GPT model. The main errors seen across all models were forgetting ERE syntax (specifically attempting to use RegEx only syntax) and logical errors (failing to catch some valid/invalid pattern). The NL Clarifier & Event-to-NL tools did seem to significantly improve the accuracy, from 0.42 to 0.61.

Impact & Future Work

The automation of ERE logic generation significantly lowers the barrier to entry for using MOP and also saves time even for experienced software engineers; the tool turns ERE generation time from hours to seconds. Future iterations will explore capabilities for other formal logics, as evidence shows that logic like Linear Temporal Logic may be easier for transformer models to interpret. We also plan to test the strength of this tool on industry runtime-verification tasks, as the testing done for this project was on Javadoc MOP specifications. Lastly, we plan to open-source all architectures and comparison tool.

Generating Runtime-Verification Specs from Natural Language



Daniel Feng

Introduction

Runtime Verification (RV) is a crucial software engineering process used to continuously monitor execution traces and catch logical violations in real-time. Essentially, RV is automated debugging for complex programs. However, translating simple English requirements into the complex formal logic required by RV frameworks makes a high barrier-of-entry to RV. To address this bottleneck, this project automates the translation of natural language (NL) software requirements into Extended Regular Expressions (ERE) for the Monitoring-Oriented Programming (MOP) framework by utilizing Large Language Models.

Novelty

While prior research has explored using LLMs to translate natural language into formal logic, existing methods either focus on logic languages other than ERE (focusing on Linear Temporal Logic instead) and don't consider the context of runtime-verification. This project introduces a pipeline targeting the nuances found in RV ERE logics. Additionally, unlike previous tools that relied on human experts to verify the accuracy of the translation tool, this approach develops an automated semantic comparison engine that rigorously defines what "equivalence" means..

Methodology

The method employs a multi-transformer architecture utilizing, consisting of an NL Clarifier (NLC), an Event Interpreter (EI), and an ERE generator (EREG). The NLC normalizes ambiguous natural language requirements and adds potential necessary context; the EI translated events in the execution trace to NL; the EREG uses the previous results to produce ERE. The prompt to the EREG was iteratively refined, and then tested across multiple Large Language Models (GPT 5.5, a base open-source model, a fine-tuned open-source model (OSM), and a non-tuned OSM) to establish the most effective generation strategy. Finally, we developed a Python engine that converts both the generated and ground-truth EREs into modified minimal deterministic finite automata (min-DFA) to mathematically prove their semantic equivalence.

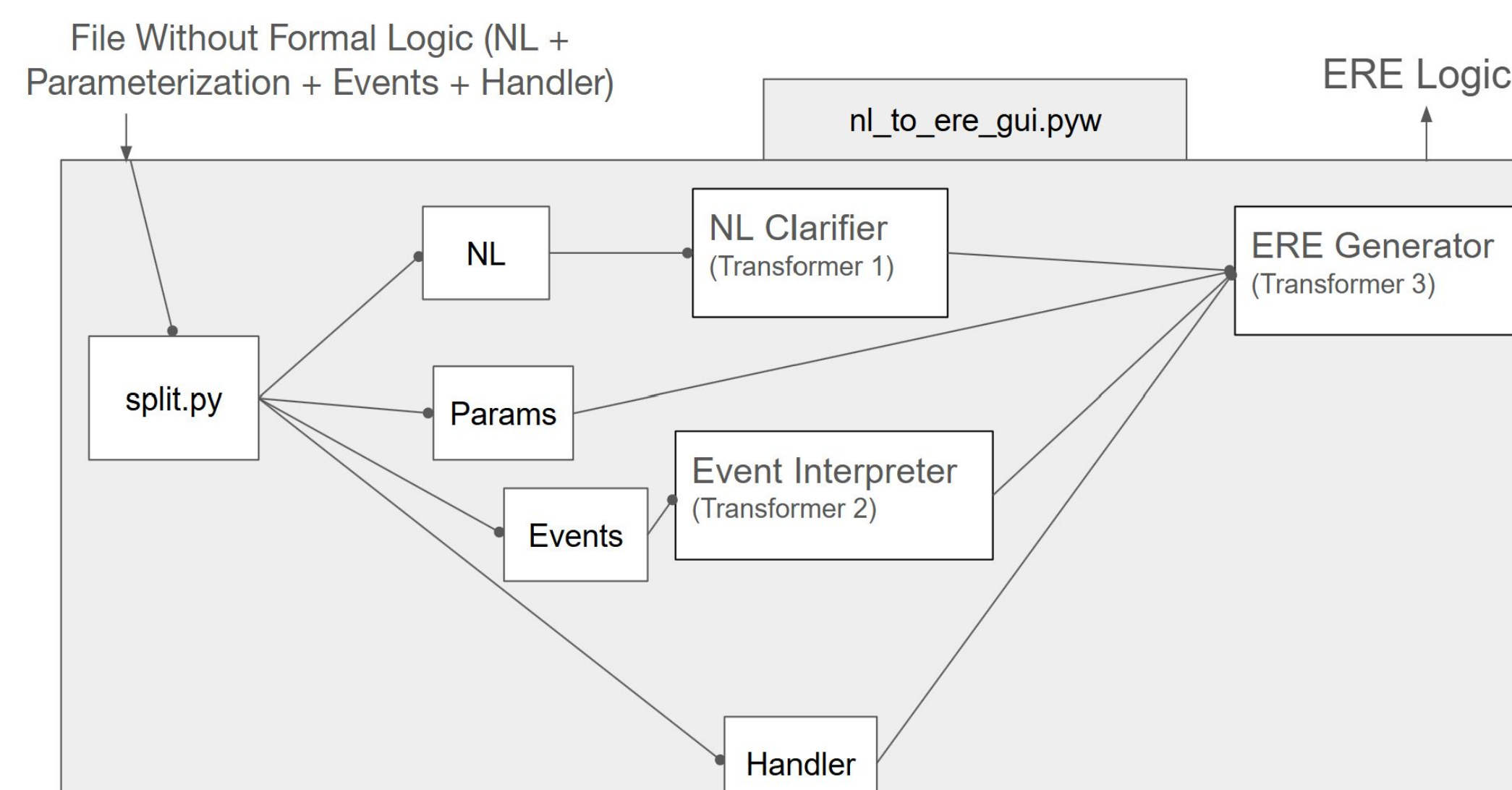


Fig. 1: Full Tool Architecture

Results

Table 2: Performance of Evaluated Models on ERE Generation

Model Configuration	Semantic Accuracy	Syntactic Accuracy
GPT 5.5 w/ Iterative Prompt	0.42	0.32
GPT 5.5 w/ Iterative Prompt & NL Clarifier & Event-to-NL	0.61	0.44
GPT-OSS w/ Iterative Prompt	0.22	0.15
GPT-OSS w/ Iterative Prompt & NL Clarifier & Event-to-NL	0.28	0.19
Fine-tuned GPT-OSS w/ Iterative Prompt & NL Clarifier & Event-to-NL	0.35	0.21

Table 1: Results of Testing

Impact & Future Work

The automation of ERE logic generation significantly lowers the barrier to entry for using MOP as well as other RV frameworks, allowing novices to use real-time large-software debugging tools. Future iterations will explore more advanced models to improve semantic accuracy and expand the system to support other formal logics like Linear Temporal Logic. Finally, the tool will be deployed within an active enterprise codebase to validate its practical efficacy against complex, proprietary industry requirements.

Generating Runtime-Verification Specs from Natural Language



Introduction

As many students hail from long-ways away, the commute to TJ can become a long and tedious task. Hundreds of students without adequate accommodations to school and back can reduce school and after school activity attendance. Facing the same problem ourselves, we developed a ridesharing system to solve this.

Novelty

No prior rideshare method exists at TJ. Our app is safe and secure for all students to use, as it ensures that all users of the app are members of the TJ Intranet before application access. While we do plan to expand past TJ eventually, the specificity of our app allows us to optimally target one community. Our system also is **completely free** for all students to use through our system **RidePoints**. This system is an incentivizing system that allows drivers to gain points which they can later use to pay for rides.

Fig. 1: Systems Architecture

Methods

As shown in *Fig. 1*, our application runs on three main servers: the frontend, the backend, the MongoDB database. The React frontend reaches out to the backend using the Axios library, which then accesses the database and AWS Lambda to retrieve information such as user data and optimal ride selection data. Our entire application is hosted on a Vercel Server in the Northern Virginia region.

Results

Our app shows the riders all available drivers once they login through ION OAuth. Each user can choose to access a list of available rides or offer up to drive other users. A Google Map overlay shows a rough estimate of where a rider should be picked up and can provide direction information for a driver. The interface is shown in *Fig. 2*.

Fig. 2: Ride confirmation + RidePoints introduction

Impact & Future Work

Our app has successfully been able to connect riders and drivers in the TJ community. With the potential to boost the number of carpools happening before and after school, TJRides has the potential to greatly reduce the number of LOVs (low-occupancy vehicles) on the road — greatly reducing the emissions caused by TJ students. TJRides can also boost attendance and participation of after school activities because of the accessibility to rides. Our future work includes developing the design elements, implementing the RidePoints system and the morning rider flow.